

From Next Token Prediction to Agency

tags: @vision @abstractions

Building LLM agents has never been easier, with API providers directly exposing tool calls as a first-class primitive and a myriad of opinionated frameworks each harnessing the API in a slightly different way. This fertile ground has led to such a proliferation of open-source projects and startups that it is now common to conflate agents with LLMs interacting with some external system. Yes, we are finally able to write adaptive workflows, or even goal-oriented programs and sophisticated coding assistants, but the result often feels stitched together. There seems to be a disconnect between the way we conceptualize and train LLMs and the way we expect intelligent agents to behave. We generally understand agency as a high-level cognitive process involving abstractions like observations, states, actions, and rewards. On the other hand, we understand underlying language models through the lens of transformers approximating a finer-grained token-level stochastic process. Throughout this post, we will refer to these two different yet coexisting perspectives as the macro-level (agentic) and micro-level (autoregressive) views.

Thankfully, both perspectives are amenable to mathematical formalization and have been explored by greater minds. Encouraged by the empirical certainty that LLM agents exist, we will formalize how an LLM can become an agent. We will draw on existing abstractions and known equivalences to construct a narrative that lifts the autoregression of symbolic sequences into a coherent theory of agentic decision-making. Without claiming any secret LLM training sauce, I hope to shed light on esoteric properties of LLM agents that follow from their mathematical nature.

We start by characterizing a Partially Observable Markov Decision Process (POMDP) as an autoregressive stochastic process over actions and observations. We use this characterization as a generator for agentic traces and derive what information is stored in a model trained to autoregressively predict sequences sampled from such a generator. Concretely, we treat turn-labeled sequences as the observable process and the POMDP as the latent mechanism that produces them. We build intuition about what LLMs would implicitly learn during pretraining if trained on an extensive corpus of agentic traces, under the simple assumption that the tokenization space aligns with the action and observation spaces.

We then relax the exact-tokenization assumption and consider agentic traces where each action and observation is composed of a sequence of symbols. We show how the original next-token formulation of the transformer, together with special turn-delimiting tokens, can express the probability of these sequences as a function of turns rather than raw token history. This gives us a clear view of two distinct problems and their connection. At the macro level, we have a turn-based POMDP with its own action, observation, and hidden state spaces. The agent observes the history of actions and observations and emits a new action; if valid, the environment updates its hidden state, emits a new observation that is appended to the history, and the process repeats. At the token level, the LLM observes a serialized representation of this history where each turn is separated by a special

token and generates a new sequence of tokens; if valid, the environment emits a new sequence that is appended to the model context, and the process repeats. The connection is now evident: the macro process exists when the micro process generates sequences recognized as valid actions by the environment. We can think of the LLM as an optimizing agent when its micro-level token choices induce high-level actions that lead toward high-reward states.

While this picture is getting complicated, this sort of problem is not novel in reinforcement learning. Think of a robotic arm controlled by a continuous controller that needs to play a video game using a keyboard. The high-level problem is the video game, while the low-level problem is the motor control of the arm. This setup is often interpreted as hierarchical reinforcement learning, where local policies generate temporally extended actions modeled as temporal options, while the high-level policy chooses which option to execute.

By the end of this post, I hope to have persuaded you that we can interpret an LLM interacting with an environment as a hierarchical POMDP, where the transformer implements micro-level controllers at the level of options that compose into a coherent high-level policy.

The POMDP Frame or When is a System Agentic?

We start by asking in what sense a dynamical system can be said to be agentic, and what latent structure can be recovered from the system’s measurable behavior when this assumption holds. Agency developed historically as a model of what separates humans from the rest of physical matter, and more recently as a model of what makes a system intelligent. Being humans, we inherently perceive a separation between what we are as a mind, the agent, and the thing-in-itself we live in, the environment. This dualism seems to clash with our understanding of reality as a local physical process, yet it is reconciled in modern science as a conditional independence, or causal, structure that appears when we measure the world at a coarse enough scale. If we zoom out sufficiently, we can see how the states inside the agent are causally independent of the states in the environment given the information that leaks in at the boundary between them. We can think of this as the agent’s perceptual channel. Conversely, we can see how the state of the world is conditionally independent of what goes on inside the agent given the information that leaks out at the boundary, which we can call the causal or action channel.

Make the physical example concrete. Imagine a room with a robotic arm at a keyboard connected to an Atari console, a Raspberry Pi running the controller, and a camera watching the Atari’s monitor. We can leave the robot in the room until it learns to play the game at the best of its capacity, the we start measuring. If all we can monitor in that room are keypress events and camera frames, those two streams are our entire window into the dynamics at this resolution. Continuous time effectively unfolds in turns: a keypress is an action, the game produces pixels as a response, and the cycle repeats.

Given these measurements, we could be tempted to build a complete bottom-up model of the room, starting from silicon steering electrons through transistors, up to the motor drivers and linkages that move the arm, and down again to the specific software implementation of the game and the hardware it runs on. That would be a daunting task, and it would change if the same behavioral input were generated by a different robot or if the game were implemented on different hardware. Thankfully, none of these details change the coarse-grained causal boundary between agent and environment. Given the keys that were pressed, the game’s next internal update does not depend on the Pi’s microstate; given the pixels that hit the camera, the Pi’s next update does not depend on the game’s microphysics. The decomposition between agent and environment gives us the structure

to describe a stochastic generator of the measurement process that is invariant to the microscopic details of the system.

In this sense, it is useful to speak about an agentic system when its measurable behavior can be decomposed into two components: actions and observations, each defining a clear causal boundary, or Markov blanket, between the environment and the agent. In this section, we formally derive the POMDP frame as a decomposition of an autoregressive stochastic process into two stationary components. This setup allows us to connect the dots with our understanding of LLMs as approximators of autoregressive stochastic processes, yielding insights into what an LLM trained on agentic traces would learn.

Agentic Traces as an Autoregressive Stochastic Process

To make our discussion formal, we start from reviewing the general language of POMDPs to define the macro-level agent-environment interaction. In practice modelling a system as a POMDP corresponds to assuming a specific decomposition of the observable process, or agentic-trace, Z into a sequence of turns $Z = \{z_1, z_2, \dots, z_T\}$, where each turn $z_t \in \mathcal{Z}$ is a tuple (a_t, o_t) of an action a_t and an observation o_t . Intuitively the action a_t in $\mathcal{A}$ defines the component of the observable turn that is causally controlled by the agent, while the observation o_t in $\mathcal{O}$ defines the component that is causally controlled by the environment conditioned on the agent behavior. The sets $\mathcal{A}$ and $\mathcal{O}$ decompose the degrees of freedom of $z \in \mathcal{Z}$ into the product $\mathcal{A} \times \mathcal{O}$.

Then we assume that the probability of the next joint turn $z_{t+1} = (a_{t+1}, o_{t+1})$ is only conditioned on the current environment and agent hidden states, respectively s_t and agent's $\hat{s}_t$, and the agent's parameters θ , defining a joint stochastic process with emissions $P(z_{t+1} | s_t, \hat{s}_t; \theta)$ whose temporal dynamics are modeled by the recurrent transition $P(s_{t+1}, \hat{s}_{t+1} | s_t, \hat{s}_t, z_{t+1}; \theta)$. Or explicitly with respect to actions and observations $P(a_{t+1}, o_{t+1} | s_t, \hat{s}_t; \theta)$. In a POMDP it is possible to further decompose this joint process into the environment's emission kernel $P(o_{t+1} | s_t, a_{t+1})$ and its emission-conditioned transition kernel, $P(s_{t+1} | s_t, a_{t+1}, o_{t+1})$, together with the agent's policy $\pi(a_{t+1} | \hat{s}_t; \theta_\pi)$ and the agent's state transition kernel $\mathcal{M}(\hat{s}_{t+1} | \hat{s}_t, a_{t+1}, o_{t+1}; \theta_\mathcal{M})$, with parameters $\theta = (\theta_\pi, \theta_\mathcal{M})$. This choice yields a unifilar latent update at the environment level: once s_t , a_{t+1} , and the realized o_{t+1} are known, the distribution over s_{t+1} is conditionally concentrated along a single causal branch consistent with that symbol.

The emission of the next turn decomposes into agent action selection and environment observation emission:

$$P(z_{t+1} | s_t, \hat{s}_t; \theta) = P(a_{t+1}, o_{t+1} | s_t, \hat{s}_t; \theta) = \pi(a_{t+1} | \hat{s}_t; \theta_\pi) \cdot P(o_{t+1} | s_t, a_{t+1}).$$

This is the macro interface where the agent proposes a_{t+1} and the environment commits to a symbol o_{t+1} that will also drive the unifilar update of its hidden state.

Given the emitted turn, both hidden states update according to their respective transition kernels:

$$P(s_{t+1}, \hat{s}_{t+1} | s_t, \hat{s}_t, z_{t+1}; \theta) = P(s_{t+1} | s_t, a_{t+1}, o_{t+1}) \cdot \mathcal{M}(\hat{s}_{t+1} | \hat{s}_t, a_{t+1}, o_{t+1}; \theta_\mathcal{M}).$$

Here the unifilar dependence of s_{t+1} on (s_t, a_{t+1}, o_{t+1}) guarantees that the latent path is synchronizable from the turn process in the sense used in computational mechanics.

Combining emission and transition dynamics, the complete joint update for each turn $z_{t+1} = (a_{t+1}, o_{t+1})$ becomes:

$$P(z_{t+1}, s_{t+1}, \hat{s}_{t+1} | s_t, \hat{s}_t; \theta) = P(z_{t+1} | s_t, \hat{s}_t; \theta) \cdot P(s_{t+1}, \hat{s}_{t+1} | s_t, \hat{s}_t, z_{t+1}; \theta).$$

Which expands to the full factorization $P(a_{t+1}, o_{t+1}, s_{t+1}, \hat{s}_{t+1} \mid s_t, \hat{s}_t; \theta)$:

$$\pi(a_{t+1} \mid \hat{s}_t; \theta_\pi) \cdot P(o_{t+1} \mid s_t, a_{t+1}) \cdot P(s_{t+1} \mid s_t, a_{t+1}, o_{t+1}) \cdot \mathcal{M}(\hat{s}_{t+1} \mid \hat{s}_t, a_{t+1}, o_{t+1}; \theta_{\mathcal{M}}).$$

For a complete trajectory $Z = \{z_1, z_2, \dots, z_T\}$ with $z_t = (a_t, o_t)$, the joint probability over all turns and hidden states given the agent parameters θ is:

$$P(Z, s_{1:T}, \hat{s}_{1:T} \mid \theta) = P(s_1, \hat{s}_1) \cdot P(z_1 \mid s_1, \hat{s}_1; \theta) \cdot \prod_{t=1}^{T-1} P(z_{t+1}, s_{t+1}, \hat{s}_{t+1} \mid s_t, \hat{s}_t; \theta).$$

For a given agent, marginalizing over the hidden states gives the observable sequence probability:

$$P(Z \mid \theta) = \sum_{s_{1:T}, \hat{s}_{1:T}} P(s_1, \hat{s}_1) \cdot P(z_1 \mid s_1, \hat{s}_1; \theta) \cdot \prod_{t=1}^{T-1} P(z_{t+1}, s_{t+1}, \hat{s}_{t+1} \mid s_t, \hat{s}_t; \theta).$$

With the latest formulation we can effectively define the likelihood of our data conditional on a generative model that can be effectively compartmentalized into a parametrized agent and a stationary environment, that is an agentic system.

Learning from a Dataset of Agentic Traces

In the previous section we showed how a partially observable dynamical system can be decomposed into two stationary parts: a fixed environment and a parameterized agent. Now I want intuition for what can be learned from data generated by that process. Assume the agent has converged to a stationary policy and we can sample a large dataset of agentic traces $Z = \{z_1, \dots, z_T\}$ with turns $z_t = (a_t, o_t)$. The question is simple: what is the informational relationship between the generator's hidden states $(s_t, \hat{s}_t)$ that produced a given z_t and the internal state $h_{\theta_G}(z_{<t})$ of a strong autoregressive predictor $G(\hat{z}_t \mid z_{<t}; \theta_G)$?

There are two standard ways to model hidden stochastic processes.

Belief over latent state. We put a probabilistic model on the world and filter observations into a belief over latent state

$$b_t(s) = P(s_t = s \mid a_{1:t}, o_{1:t}).$$

After a new turn (a_{t+1}, o_{t+1}) the belief updates with Bayes

$$b_{t+1}(s') \propto \sum_s b_t(s) P(o_{t+1} \mid s, a_{t+1}) P(s' \mid s, a_{t+1}, o_{t+1}).$$

Under our setup the environment update is unifilar at the turn scale. Once (s_t, a_{t+1}, o_{t+1}) are fixed, the next branch is unique almost surely. This gives synchronization: as turns accumulate, the belief concentrates along the causal branch consistent with the observations.

Predictive state from data. We can skip latent variables and learn a predictive state representation q_t that summarizes the history by what it implies for future observations under future actions. For finite horizons k ,

$$P(o_{t+1:t+k} \mid a_{t+1:t+k}, z_{\leq t}) = P(o_{t+1:t+k} \mid a_{t+1:t+k}, q_t).$$

In practice it is enough to match the one step case and update q_{t+1} deterministically from (q_t, a_{t+1}, o_{t+1}) . This is the world model view trained by next observation prediction.

These two views meet at the minimal predictive presentation. Among optimal nonlinear predictors of a stationary process, the ε -machine (and with inputs, the ε -transducer) collects histories that induce the same conditional futures into the same causal state. In the finite-state stationary case we care about, and under unifilarity, the history-based predictor is equivalent to a minimal unifilar generator. So the smallest sufficient predictive state you can build from data coincides with the smallest unifilar presentation of the turn process.

What does a strong autoregressive model have to encode to predict our traces. Recall the emission decomposition

$$P(z_{t+1} \mid s_t, \hat{s}_t) = \pi(a_{t+1} \mid \hat{s}_t) \cdot P(o_{t+1} \mid s_t, a_{t+1}).$$

A good next turn predictor trained on Z must internalize a predictive latent that is sufficient for after factors. Its internal state $h_{\theta_G}(z_{\leq t})$ needs to behave like a predictive state q_t : enough information about the history to produce the next action and to forecast the next observation in response to that action. Under unifilarity and stationarity this predictive latent aligns with the causal state of the ε -transducer at the level of turns.

Finally, what does this mean for a transformer trained on sequences of turns. In practice each turn is a tuple $z_t = (a_t, o_t)$. The natural training target is two step: first predict the action, then predict the observation given that action and the history. Concretely the model estimates

$$P(a_{t+1} \mid z_{\leq t}) \quad \text{and} \quad P(o_{t+1} \mid a_{t+1}, z_{\leq t}).$$

The internal state right after emitting a_{t+1} is a sufficient statistic of the agent side $\hat{s}_t$ for the purpose of the policy. The internal state right after receiving o_{t+1} is a sufficient statistic of the environment branch s_{t+1} for the purpose of predicting the next observation. This boundary effect also explains a mild separability we observe in practice. A single shared head can support both factors, but the logits factor by conditioning order: action logits depend mostly on the agent slice of the predictive state, observation logits condition further on the realized action and align with the environment slice. That is the concrete story of what a transformer trained on these sequences has learned. It has learned a predictive state that simultaneously supports an action policy and a world model at the level of turns, and it updates that state at the two natural boundaries of the turn, first at action time and then at observation time.

Reward Maximization and Goal Directedness

When a reward function $R : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ exists, we define the objective functional:

$$J(\theta) = \mathbb{E}_{Z \sim P(Z|\theta)} \left[\sum_{t=1}^T R(s_t, a_t) \right]$$

Learning then becomes a dynamical process over the parameter space, defined by the update operator $\Phi : \Theta \rightarrow \Theta$ such that $\theta_{k+1} = \Phi(\theta_k)$, where Φ could be gradient ascent:

$$\Phi(\theta) = \theta + \eta \nabla_{\theta} J(\theta)$$

or any other optimization procedure. The reinforcement learning problem seeks the fixed point $\theta^* \in \Theta$ satisfying:

$$\theta^* = \lim_{k \rightarrow \infty} \Phi^{(k)}(\theta_0) = \arg \max_{\theta \in \Theta} J(\theta)$$

Since $\theta = (\theta_\pi, \theta_{\mathcal{M}})$, we can make explicit that the optimization operates solely over the agent’s policy and world model parameters:

$$(\theta_\pi^*, \theta_{\mathcal{M}}^*) = \arg \max_{\theta_\pi, \theta_{\mathcal{M}}} \mathbb{E}_{Z \sim P(Z|\theta_\pi, \theta_{\mathcal{M}})} \left[\sum_{t=1}^T R(s_t, a_t) \right]$$

where the expectation is taken with respect to trajectories generated by the agent’s policy $\pi(\cdot | \cdot; \theta_\pi)$ and state updates $\mathcal{M}(\cdot | \cdot; \theta_{\mathcal{M}})$, while the environment dynamics $P(o_{t+1} | s_t, a_{t+1})$ and $P(s_{t+1} | s_t, a_{t+1}, o_{t+1})$ remain fixed and unparameterized. Here $\Phi^{(k)}$ denotes the k -fold composition of Φ , assuming convergence from initial parameters θ_0 . In practice, we settle for local optima or stationary points where $\|\theta_{k+1} - \theta_k\| < \epsilon$ for some tolerance $\epsilon > 0$.

The world model with parameters $\theta_{\mathcal{M}}$ learns to approximate this distribution. Let $\hat{P}(o_{t+1} | a_{t+1}, \hat{s}_t; \theta_{\mathcal{M}})$ be the model’s prediction given its internal state $\hat{s}_t$ (which encodes the belief). The learning objective becomes minimizing the KL divergence:

$$\mathcal{L}_{\mathcal{M}}(\theta_{\mathcal{M}}) = \mathbb{E}_{h_t, a_{t+1}} \left[D_{KL} \left(P(o_{t+1} | a_{t+1}, h_t) \| \hat{P}(o_{t+1} | a_{t+1}, \hat{s}_t; \theta_{\mathcal{M}}) \right) \right]$$

This simplifies to the cross-entropy loss in practice:

$$\mathcal{L}_{\mathcal{M}}(\theta_{\mathcal{M}}) = -\mathbb{E}_{h_t, a_{t+1}, o_{t+1}} \left[\log \hat{P}(o_{t+1} | a_{t+1}, \hat{s}_t; \theta_{\mathcal{M}}) \right]$$

Given an optimal (or sufficiently accurate) world model $\theta_{\mathcal{M}}^*$, the policy optimization becomes:

$$\theta_\pi^* = \arg \max_{\theta_\pi} \mathbb{E}_{Z \sim P(Z|\theta_\pi, \theta_{\mathcal{M}}^*)} \left[\sum_{t=1}^T R(s_t, a_t) \right]$$

This decomposition reveals a fundamental principle: accurate state estimation (via the world model) is a prerequisite for optimal decision-making (via the policy). The world model provides the informational substrate upon which the policy operates, transforming partial observations into belief states that support reward-maximizing action selection.

From Next-Token Prediction to Decision-Making

In the previous section we derived the macro-level decision-making problem as a decomposition of the resulting stochastic process connecting the observable autoregressive dynamics to the underlying agent–environment relationship. The one caveat is that we were reasoning directly in terms of autoregression over high-level symbols, implicitly assuming that the transformer’s token space coincides with the union of the alphabets $\mathcal{A}$ and $\mathcal{O}$, together with their Cartesian product $\mathcal{A} \times \mathcal{O}$. In this section we relax that assumption. The underlying process operates on a much finer-grained alphabet Σ whose only algebraic structure is right-side concatenation, and the coarser alphabet of turns $\mathcal{Z}$ is induced as a quotient space defined by special delimiter tokens. This construction allows the next-token process to realize an effective agent–environment interface where each completed

segment of tokens corresponds to a full turn (a_t, o_t) in the macro POMDP. Crucially, this turn-level view aligns with the **unifilar** structure of the environment, where each emitted observation o_{t+1} simultaneously marks the completion of a turn and determines the unique causal branch of the environment’s latent transition.

We start from the basic autoregressive formulation of a language model, which defines a probability distribution over token sequences of length T conditional on the semi-infinite prefix of previously generated tokens:

$$P_\theta(\tau_{1:T}) = \prod_{t=1}^T P_\theta(\tau_t \mid \tau_{<t})$$

where τ_t denotes the token generated at position t , and $\tau_{<t}$ represents the prefix of previously generated tokens. This defines a left-to-right stochastic process that samples each token sequentially, conditioned on the textual history. In POMDP terms, this autoregressive process defines the *micro-dynamics* of the agent’s internal policy π , acting within the observation space defined by its evolving textual context.

From Tokens to Turns to Actions and Observations

To lift this token-level process into a decision-making framework compatible with the POMDP perspective, we segment the token stream into turns using special control tokens inserted during assistant post-training. These tokens provide explicit syntactical boundaries between:

1. the agent’s generations (tool calls or textual reasoning),
2. the environment’s responses, and
3. the user’s requests.

Each turn thus corresponds to a higher-level event in the stochastic process, marking a complete interaction step between the agent and the environment. This segmentation allows us to reinterpret the autoregressive process in terms of turn-level sequences, which naturally play the role of actions and observations within the POMDP framework.

Let each turn be the contiguous block of tokens

$$\text{turn}_t = (\tau_{k_t}, \dots, \tau_{k_{t+1}-1})$$

ending with a distinguished delimiter token EOT. We can then define the induced distribution over turns as

$$P_\theta(\text{turn}_{1:N}) = \prod_{t=1}^N P_\theta(\text{turn}_t \mid \text{turn}_{<t})$$

Each conditional distribution represents the behavioral policy of the agent at the turn level. It describes how the model generates the next complete action sequence—its next decision—conditioned on its full observation history.

Stopping times and measurability

Let $(\mathcal{F}_k)_{k \geq 0}$ be the natural filtration generated by the token sequence, where $\mathcal{F}_k = \sigma(\tau_{1:k})$. Fix the distinguished delimiter $\text{EOT} \in \Sigma$. Define the stopping times for turn boundaries by

$$\kappa_1 = \inf\{k \geq 1 : \tau_k = \text{EOT}\}, \quad \kappa_{t+1} = \inf\{k > \kappa_t : \tau_k = \text{EOT}\}, \quad \kappa_0 := 0.$$

Each κ_t is an $(\mathcal{F}_k)$ -stopping time, and the t -th completed turn block is

$$\text{turn}_t = (\tau_{\kappa_{t-1}+1}, \dots, \tau_{\kappa_t}).$$

The mapping from a token path to its corresponding finite sequence of completed turns is therefore measurable with respect to the filtration. This formalizes that the end of a turn is determined by information available at the stopping time κ_t . Under the unifilar formulation of the environment, the realized observation o_t that closes turn t uniquely determines the next environment branch, implying that the latent generator of the observable turn process can be taken to be the environment state s_t itself. This property guarantees **synchronizability**: the macro process over turns is informationally equivalent to the hidden-state process (s_t) .

Quotient mapping by independent parsers

Let Σ be the token alphabet and Σ^* the set of finite token strings. Define two token-level parsers

$$\rho_{\mathcal{A}} : \Sigma^* \rightarrow \mathcal{A}, \quad \rho_{\mathcal{O}} : \Sigma^* \rightarrow \mathcal{O}$$

that independently extract the action and the observation components from a completed turn string. Define the joint parser

$$\rho_{\mathcal{Z}}(x) = (\rho_{\mathcal{A}}(x), \rho_{\mathcal{O}}(x)) \in \mathcal{A} \times \mathcal{O}.$$

Introduce the equivalence relation on Σ^*

$$x \sim y \iff \rho_{\mathcal{Z}}(x) = \rho_{\mathcal{Z}}(y).$$

The quotient space of turn semantics is then

$$\mathcal{Z} = \Sigma^* / \sim, \quad [z] = \{x \in \Sigma^* : \rho_{\mathcal{Z}}(x) = z\}, \quad z = (a_t, o_t) \in \mathcal{A} \times \mathcal{O}.$$

For any completed turn block $x = \text{turn}_t$ we write $z_t = \rho_{\mathcal{Z}}(x)$ and $x \in [z_t]$.

Class-marginalization from tokens to turns

The token model induces a turn-level distribution by marginalizing over all token realizations in the equivalence class of the parsed turn. We restrict this marginalization to **prefix-free** token strings that contain exactly one delimiter at the end and none internally. Let $L_{z_t} \subseteq [z_t]$ denote this parser-recognized language of completed turns. Then

$$P_\theta(z_t \mid \text{turn}_{<t}) = \sum_{x \in L_{z_t}} P_\theta(x \mid \text{turn}_{<t}),$$

where for any $x = (\tau_m, \dots, \tau_n)$ ending with the delimiter,

$$P_\theta(x \mid \text{turn}_{<t}) = \prod_{k=m}^n P_\theta(\tau_k \mid \tau_{<k}).$$

This recovers the desired turn-level process from the micro-level autoregression while ensuring that each turn is counted exactly once and ends at the first valid delimiter.

Concatenation compatibility and class projection

Define the projection

$$q : \Sigma^* \rightarrow \mathcal{Z}, \quad q(x) = [\rho_{\mathcal{Z}}(x)].$$

Let $\sim$ denote token concatenation and let $x \parallel y$ denote concatenation along turn boundaries, that is, x ends at a delimiter and y begins immediately after. Then concatenation followed by class projection is compatible with turn concatenation modulo delimiters:

$$q(x \parallel y) = q(x) \odot q(y),$$

where $\odot$ is the induced concatenation of equivalence classes at the turn level. In particular, if $x \in [z_t]$ and $y \in [z_{t+1}]$ with both ending at delimiters, then $x \parallel y \in [z_t] \odot [z_{t+1}]$. This factorization justifies composing probabilities turn by turn, since the projection q respects the segmentation defined by the stopping times κ_t .

Admissibility and semantic support

The parser-induced turn space interacts with the environment through admissibility constraints. For each environment state s_t , define the admissible class set

$$\mathcal{Z}_{\text{env}}(s_t) \subseteq \mathcal{Z},$$

and the corresponding semantic indicator

$$\mathbb{I}_{\text{env}}(x, s_t) = \mathbb{I}[q(x) \in \mathcal{Z}_{\text{env}}(s_t)].$$

In practice, $\mathcal{Z}_{\text{env}}(s_t)$ is realized by a state-conditioned recognizer $\mathcal{G}(s_t)$ defining the language of admissible token sequences for that environment state. The recognizer constrains the micro policy

during decoding, ensuring that only syntactically valid and executable actions are produced. The induced set of admissible high-level actions is then

$$\mathcal{A}_{\text{env}}(s_t) = \{a : \exists o \text{ s.t. } (a, o) \in \mathcal{Z}_{\text{env}}(s_t)\},$$

which aligns the grammar-constrained token policy with the reinforcement learning action space.

Induced macro kernels from micro dynamics

We now return to the parameterization $\theta = (\theta_\pi, \theta_{\mathcal{M}})$ introduced in the POMDP formulation. At the token level, the autoregressive model defines a generator

$$\sigma_\theta(\tau_k \mid \tau_{<k}) = P_\theta(\tau_k \mid \tau_{<k}),$$

whose path probability for a completed turn block $x = (\tau_m, \dots, \tau_n)$ ending at the delimiter is

$$P_\theta(x \mid \text{turn}_{<t}) = \prod_{k=m}^n \sigma_\theta(\tau_k \mid \tau_{<k}).$$

Through the equivalence relation $x \sim y$ induced by the joint parser $\rho_{\mathcal{Z}}$, we recover the class-marginal turn distribution

$$P_\theta(z_t \mid \text{turn}_{<t}, s_t) = \sum_{x \in L_{z_t}} P_\theta(x \mid \text{turn}_{<t}) \mathbb{I}_{\text{env}}(x, s_t).$$

The induced process over turns then defines the macro-level transition kernel

$$P_\theta(z_{t+1}, s_{t+1}, \hat{s}_{t+1} \mid s_t, \hat{s}_t) = P_\theta(z_{t+1} \mid s_t, \hat{s}_t) \cdot P(s_{t+1}, \hat{s}_{t+1} \mid s_t, \hat{s}_t, z_{t+1}),$$

where under the emission-conditioned (unifilar) factorization we have

$$P_\theta(z_{t+1} \mid s_t, \hat{s}_t) = \pi_\theta(a_{t+1} \mid \hat{s}_t; \theta_\pi) \cdot P(o_{t+1} \mid s_t, a_{t+1}),$$

and equivalently, when marginalizing over the agent’s belief state,

$$P_\theta(z_{t+1} \mid b_t, \hat{s}_t) = \pi_\theta(a_{t+1} \mid \hat{s}_t; \theta_\pi) \sum_{s_t} b_t(s_t) P(o_{t+1} \mid s_t, a_{t+1}).$$

This expression mirrors the emission decomposition of the macro POMDP but now explicitly under the unifilar transition structure. Each factor arises directly from the token-level generator σ_θ through the quotient mapping, meaning that the same parameters θ induce both the **macro policy** π_θ and the **macro world model** $\mathcal{M}_\theta$. This identification closes the loop between the micro and macro views: the low-level token generator σ_θ produces a synchronized, unifilar process whose class-marginals implement the high-level policy π_θ and transition model $\mathcal{M}_\theta$.